Step-by-Step: .NET 10 MAUI (C#) for iOS and Android with VS Code on a MacBook Air M3

A hands-on, command-by-command guide to set up and run a **.NET 10 MAUI** app for **iOS and Android** using **Visual Studio Code** on an **Apple Silicon (M3)** MacBook Air.

Every step lists the exact commands to run, in order, plus how to verify each one worked. Commands are for the **zsh** shell (the macOS default).

> **Read this first.** Apple's build tools only run on macOS, so the Mac is the single source of truth for iOS. Android builds also work natively on the Mac. This guide is **Mac-only with VS Code** — no Windows PC and no Visual Studio are required.
>
> **Versions move fast.** .NET workloads are tightly coupled to a specific **Xcode** version. A mismatch produces errors like *"This version of .NET for iOS (X) requires Xcode Y. The current version of Xcode is Z."* Always confirm the current pairing in Section 10 before installing.

Table of contents

1. What you are building and how the pieces fit
2. Before you start: accounts, disk, and terminal
3. Step 1 — Install Homebrew
4. Step 2 — Install the .NET 10 SDK (arm64)
5. Step 3 — Install the MAUI workloads
6. Step 4 — Install Xcode and the command-line tools (iOS)
7. Step 5 — Install the Java JDK (Android)
8. Step 6 — Install the Android SDK and an emulator (Android)
9. Step 7 — Install VS Code and the .NET MAUI extension
10. Step 8 — Configure the environments from VS Code
11. Step 9 — Create your first MAUI app
12. Step 10 — Run and debug on the iOS Simulator
13. Step 11 — Run and debug on the Android Emulator
14. Step 12 — Run on physical devices
15. Step 13 — Hot Reload and day-to-day workflow
16. Step 14 — Add a Git repo and build your first APK/IPA
17. Version compatibility matrix
18. Command cheat sheet
19. Troubleshooting
20. Sources

---

1. What you are building and how the pieces fit

.NET MAUI (Multi-platform App UI) lets one C#/XAML codebase target iOS, Android, macOS, and Windows. On a Mac, you can build and run:

- **Android** — natively on the Mac (no extra machine needed).
- **iOS** — natively on the Mac, using Xcode's toolchain and the iOS Simulator.

Piece	What it does	Installed in
.NET 10 SDK	Compiles C#; provides `dotnet`	Step 2
MAUI + platform workloads	Adds iOS/Android/Mac Catalyst build targets	Step 3
Xcode + command-line tools	Apple SDKs, code signing, iOS Simulator	Step 4
Java JDK (OpenJDK 17)	Required to build Android	Step 5
Android SDK + emulator	Android build tools and a virtual device	Step 6
VS Code + .NET MAUI extension	Editor, IntelliSense, debug, Hot Reload	Step 7

Golden rule for Apple Silicon: every download must be the **arm64 / Apple Silicon** build, never Intel/x64.

---

2. Before you start: accounts, disk, and terminal

Hardware and OS

- MacBook Air M3 (Apple Silicon, `arm64`).
- macOS up to date. Recent Xcode releases require a recent macOS (for example, macOS 26.2+ "Tahoe" for Xcode 26.x). Update macOS **before** installing Xcode.

Disk space — budget generously. Xcode plus components is roughly 15–40 GB; each iOS simulator runtime is several GB; the Android SDK, emulator images, and .NET workloads add more. On a 256 GB machine, keep **60–80 GB free**. Check with:

`bash df -h /`

Accounts

- A free **Apple ID** — enough for Xcode and the iOS Simulator.
- A **paid Apple Developer Program membership** — only needed for physical iOS devices and App Store publishing. Simulator work does not need it.
- A **Microsoft account with an active Visual Studio subscription** — the **C# Dev Kit** extension (installed automatically with the MAUI extension) requires it.

Open a terminal (Terminal.app or iTerm). Confirm your shell and architecture:

`bash echo $SHELL uname -m # must print: arm64 sw_vers # macOS version`

If `uname -m` prints `x86_64`, you are running under Rosetta — make sure your terminal is the native arm64 app before continuing.

---

3. Step 1 — Install Homebrew

Homebrew simplifies installing the JDK, Mono, and other tools.

```
bash /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

On Apple Silicon, Homebrew installs to `/opt/homebrew`. Add it to your PATH and load it:

```
bash echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> ~/.zprofile eval "$(/opt/homebrew/bin/brew shellenv)"
```

Verify:

```
bash brew --version which brew # expect: /opt/homebrew/bin/brew
```

---

4. Step 2 — Install the .NET 10 SDK (arm64)

Option A — official installer (recommended). Download the **.NET 10 SDK** for **macOS Apple Silicon** from https://dotnet.microsoft.com/download/dotnet/10.0 and run the `.pkg`. It installs to `/usr/local/share/dotnet` and adds `dotnet` to your PATH.

Option B — Homebrew:

```
bash brew install --cask dotnet-sdk
```

Verify (open a new terminal first so PATH refreshes):

```
bash dotnet --version # e.g. 10.0.401 dotnet --info # confirm "Architecture: arm64" and "OS Name: Mac OS X" dotnet --list-sdks
```

If you see `command not found: dotnet`, open a **new** terminal window (or run `source ~/.zprofile`) so the updated PATH is picked up.

> **Multiple SDKs:** if a project contains a `global.json`, the SDK version it pins must be installed. Check `global.json` and `dotnet --list-sdks` if a build complains.

---

5. Step 3 — Install the MAUI workloads

The SDK alone does not include MAUI. Install the workloads (elevated permissions are required on macOS):

```
bash sudo dotnet workload install maui
```

This brings in the iOS, Android, Mac Catalyst, and MAUI workloads together. If you prefer to be explicit, you can name them:

```
bash sudo dotnet workload install maui ios android maccatalyst
```

Verify:

```bash dotnet workload list
```

Expect entries for: maui, ios, android, maccatalyst

dotnet workload --info ```

Pin a workload set (recommended for reproducibility and Xcode matching). Confirm your SDK version first, then install the matching workload set:

`bash dotnet --version # e.g. 10.0.401 sudo dotnet workload install maui --version 10.0.401.1`

Keep workloads current:

`bash sudo dotnet workload update`

> **Why this matters:** the `ios` / `maccatalyst` workload is built for **one specific Xcode**. If the workload expects a different Xcode than you have, iOS builds fail. See Section 17 to resolve it.

---

6. Step 4 — Install Xcode and the command-line tools (iOS)

6.1 Install Xcode

Preferred: install Xcode from the **Mac App Store** (search "Xcode").

If you need a specific Xcode version to match your MAUI workload, download it from https://developer.apple.com/download/all/ (Apple ID required) and manage multiple versions with `xcodes`:

`bash brew install xcodesorg/made/xcodes xcodes list # see available versions xcodes install 26.6 # install a specific version xcodes select 26.6 # make it the active Xcode`

6.2 Launch Xcode once

Open Xcode from Applications and let it install additional components and accept the licence. Then, in a terminal:

`bash xcode-select --install # standalone CLT (if prompted) sudo xcode-select --switch /Applications/Xcode.app/Contents/Developer sudo xcodebuild -runFirstLaunch # accept licence non-interactively`

Verify:

`bash xcodebuild -version # e.g. Xcode 26.6 xcode-select -p # e.g. /Applications/Xcode.app/Contents/Developer`

In **Xcode > Settings > Locations**, confirm **Command Line Tools** points at your Xcode installation.

6.3 Install an iOS Simulator runtime

In **Xcode > Settings > Components**, install the **iOS Simulator** runtime you need. Or from the command line:

`bash xcodebuild -downloadPlatform iOS xcrun simctl list runtimes # confirm the runtime appears`

6.4 Create and boot a simulator

```bash xcrun simctl list devicetypes # get exact device-type identifiers xcrun simctl list runtimes # get exact runtime identifiers

xcrun simctl create "iPhone 17" "com.apple.CoreSimulator.SimDeviceType.iPhone-17" \ "com.apple.CoreSimulator.SimRuntime.iOS-26-0"

xcrun simctl boot "iPhone 17" open -a Simulator # show the Simulator window ```

> Identifiers change between Xcode versions — always copy the exact strings from `simctl list`.

---

## 7. Step 5 — Install the Java JDK (Android)

Android builds need a JDK. Microsoft OpenJDK **17** is the supported choice.

`bash brew install --cask microsoft-openjdk@17`

Point `JAVA_HOME` at it and persist that for future shells:

`bash export JAVA_HOME=$(/usr/libexec/java_home -v 17) echo 'export JAVA_HOME=$(/usr/libexec/java_home -v 17)' >> ~/.zprofile`

**Verify:**

`bash echo $JAVA_HOME # e.g. /Library/Java/JavaVirtualMachines/microsoft-17.jdk/Contents/Home "$JAVA_HOME/bin/java" -version`

---

## 8. Step 6 — Install the Android SDK and an emulator (Android)

The easiest way to install the Android SDK is the MAUI/Android **`InstallAndroidDependencies`** MSBuild target. Create a throwaway MAUI project to drive it (you can delete it afterwards), or run it against your real project later.

```bash cd ~ dotnet new maui -n MauiSetup cd MauiSetup

dotnet build -t:InstallAndroidDependencies \ -f net10.0-android \ -p:AndroidSdkDirectory="$HOME/Library/Android/sdk" \ -p:JavaSdkDirectory="$JAVA_HOME" \ -p:AcceptAndroidSDKLicenses=True ```

> Use **absolute paths** — `~` is not expanded inside these MSBuild arguments. Avoid spaces and non-ASCII characters in the paths.

Persist the Android SDK location:

`bash echo 'export ANDROID_HOME="$HOME/Library/Android/sdk"' >> ~/.zprofile export ANDROID_HOME="$HOME/Library/Android/sdk"`

### 8.1 Install the emulator and an arm64 system image

On Apple Silicon you must use an `arm64-v8a` system image (x86_64 images run poorly or not at all).

```bash cd "$ANDROID_HOME/cmdline-tools/latest/bin"

./sdkmanager --install "emulator" ./sdkmanager --install "platform-tools" ./sdkmanager --install "system-images;android-35;google_apis;arm64-v8a"

./sdkmanager --licenses # accept any remaining licences ```

### 8.2 Create and start an emulator (AVD)

```bash cd "$ANDROID_HOME/cmdline-tools/latest/bin"

./avdmanager list device # list device profiles ./avdmanager create avd -n Pixel_API35 \ -k "system-images;android-35;google_apis;arm64-v8a" -d pixel_9 ```

Start it (either via the emulator binary or VS Code's device picker):

`bash "$ANDROID_HOME/emulator/emulator" -list-avds "$ANDROID_HOME/emulator/emulator" -avd Pixel_API35`

**Verify the toolchain:**

`bash "$ANDROID_HOME/platform-tools/adb" devices # your emulator should be listed`

---

## 9. Step 7 — Install VS Code and the .NET MAUI extension

### 9.1 Install VS Code

`bash brew install --cask visual-studio-code`

Or download from https://code.visualstudio.com.

### 9.2 Install the .NET MAUI extension

`bash code --install-extension ms-dotnettools.dotnet-maui`

This automatically installs its dependencies, **C#** and **C# Dev Kit**. Alternatively, search for **".NET MAUI"** in the Extensions view and click **Install**.

### 9.3 Sign in

C# Dev Kit requires a **Microsoft account with an active Visual Studio subscription**. In VS Code, open the **Accounts** menu (bottom-left) and sign in with Microsoft.

**Verify from the terminal that the CLI can see everything:**

`bash code --list-extensions | grep -i maui dotnet workload list`

---

## 10. Step 8 — Configure the environments from VS Code

The MAUI extension ships a guided walkthrough that detects and fixes environment problems.

1. Open the Command Palette (`Cmd+Shift+P`) and run **Welcome: Open Walkthrough…** → **Get Started with .NET MAUI**.
2. If the setup steps are not shown, enable them: **Settings → Extensions → .NET MAUI → check** `Maui > Configuration: Improved Setup Experience`, then reopen the walkthrough.
3. Work through each step: - Connect your account to the **C# Dev Kit** extension. - **Set up your .NET environment.** - **Set up your .NET MAUI environment.** - **Set up your iOS and macOS environment** (only visible on a Mac). - **Set up your Android environment.**

You can also refresh each environment manually from the Command Palette:

- **.NET MAUI: Configure Apple → Refresh Apple environment**
- **.NET MAUI: Configure Android → Refresh Android environment**

Address any errors the walkthrough reports before moving on.

**Full environment sanity check (terminal):**

```
bash dotnet --info dotnet workload list xcodebuild -version xcode-select -p xcrun simctl list runtimes echo $JAVA_HOME "$ANDROID_HOME/platform-tools/adb" devices
```

---

11. Step 9 — Create your first MAUI app

11.1 Scaffold from the terminal

```
bash cd ~/Projects # or wherever you keep code dotnet new maui -n MyMauiApp cd MyMauiApp
```

Or from VS Code: **Command Palette → .NET MAUI: New Project…**, give it a name, and choose a folder.

11.2 Restore and do a first build

```
bash dotnet restore dotnet build -f net10.0-ios -p:RuntimeIdentifier=iossimulator-arm64 dotnet build -f net10.0-android
```

A successful build confirms the SDK, workloads, Xcode, and Android SDK are all wired up.

11.3 Open in VS Code

```
bash code .
```

Wait for the C# Dev Kit to finish loading the project. In the status bar (bottom), the **curly-braces** `{ }` button is the **Debug Target** picker — you will use it to choose an iOS Simulator or an Android Emulator.

---

12. Step 10 — Run and debug on the iOS Simulator

12.1 From VS Code (recommended)

1. Boot the simulator first: `open -a Simulator` (or `xcrun simctl boot "iPhone 17"`).
2. In the status bar, click the `{ }` (Debug Target) button and choose your **iOS Simulator**.
3. Press **F5** (or **Run → Start Debugging**).

VS Code builds the app, installs it on the simulator, and attaches the debugger. Set breakpoints in your C# and step through as normal.

12.2 From the terminal

`bash dotnet build -t:Run -f net10.0-ios -p:RuntimeIdentifier=iossimulator-arm64`

To target a specific simulator by UDID:

`bash xcrun simctl list devices | grep Booted dotnet build -t:Run -f net10.0-ios \ -p:_DeviceName=:v2:udid=<SIMULATOR_UDID>`

12.3 Verify the app is running

`bash xcrun simctl listapps booted | grep -i mymauiapp`

---

13. Step 11 — Run and debug on the Android Emulator

13.1 Start the emulator

`bash "$ANDROID_HOME/emulator/emulator" -avd Pixel_API35 & "$ANDROID_HOME/platform-tools/adb" devices # wait until it shows "device"`

13.2 From VS Code (recommended)

1. In the status bar, click the `{ }` (Debug Target) button and choose your **Android Emulator** (or a connected physical device).
2. Press **F5**.

13.3 From the terminal

`bash dotnet build -t:Run -f net10.0-android`

13.4 Useful Android commands

```bash
# Install an already-built APK
"$ANDROID_HOME/platform-tools/adb" install -r \ ./bin/Debug/net10.0-android/com.companyname.mymauiapp-Signed.apk

# Stream device logs (great for crash triage)
"$ANDROID_HOME/platform-tools/adb" logcat | grep -i mymauiapp

# Force-stop / uninstall
"$ANDROID_HOME/platform-tools/adb" shell am force-stop com.companyname.mymauiapp
"$ANDROID_HOME/platform-tools/adb" uninstall com.companyname.mymauiapp
```

---

14. Step 12 — Run on physical devices

14.1 Physical Android device

1. On the device: **Settings → About phone → tap Build number 7 times** to unlock **Developer options**.
2. Enable **USB debugging**, connect via USB, and accept the prompt on the device.
3. Verify and deploy:

```
bash "$ANDROID_HOME/platform-tools/adb" devices dotnet build -t:Run -f net10.0-android -p:AdbTarget=-d
```

14.2 Physical iOS device

Running on a real iPhone/iPad needs a **paid Apple Developer Program** membership, a signing certificate, and a provisioning profile.

1. Connect the device and register its UDID at https://developer.apple.com/account/resources/devices/list (or let Xcode register it under **Xcode → Window → Devices and Simulators**).
2. Create an **Apple Development** certificate: **Xcode → Settings → Accounts →** your team **→ Manage Certificates… → + → Apple Development**. This installs the certificate and its private key into **Keychain Access**.
3. For automatic provisioning, set the team in the project file:

```
xml <PropertyGroup Condition="$([MSBuild]::GetTargetPlatformIdentifier('$(TargetFramework)')) == 'ios'"> <CodesignKey>Apple Development</CodesignKey> <CodesignProvision>Automatic</CodesignProvision> <CodesignTeamId>YOUR_TEAM_ID</CodesignTeamId> </PropertyGroup>
```

4. Build and run:

```
bash dotnet build -t:Run -f net10.0-ios -p:RuntimeIdentifier=ios-arm64
```

Verify signing identities on the Mac:

```
bash security find-identity -v -p codesigning ls ~/Library/MobileDevice/Provisioning\ Profiles/
```

> **Back up your identity.** Export the certificate and private key from **Keychain Access** as a `.p12` and store the file and password safely. If the private key is lost, the certificate must be revoked and every device re-provisioned.

---

15. Step 13 — Hot Reload and day-to-day workflow

- With the app running in debug, edit C# or XAML and save — **Hot Reload** applies most changes without restarting.
- Apply changes explicitly: **Command Palette → .NET MAUI: Apply Hot Reload Changes** (`Cmd+Shift+Enter` by default), or the lightning-bolt icon.

- XAML edits usually reload instantly; some changes (new types, resources, app startup) need a full rebuild.
- Switch target platforms anytime with the `{ }` Debug Target picker.
- Set breakpoints and inspect variables exactly as with any C# project.

Everyday loop:

`bash dotnet build -f net10.0-android # quick compile check dotnet build -t:Run -f net10.0-android # run on the emulator dotnet build -t:Run -f net10.0-ios -p:RuntimeIdentifier=iossimulator-arm64`

---

16. Step 14 — Add a Git repo and build your first APK/IPA

16.1 Initialise source control

`bash git init curl -fsSL https://raw.githubusercontent.com/github/gitignore/main/VisualStudio.gitignore \ -o .gitignore git add . git commit -m "Initial commit: .NET 10 MAUI app"`

16.2 Build a release Android APK / AAB

```bash

# Debug APK (quick, unsigned for distribution)

dotnet build -f net10.0-android -c Release

# Play Store bundle (.aab) and signed APK

dotnet publish -f net10.0-android -c Release \ -p:AndroidPackageFormat=aab \ -p:AndroidKeyStore=true \ -p:AndroidSigningKeyStore=myrelease.keystore \ -p:AndroidSigningKeyAlias=myalias \ -p:AndroidSigningKeyPass=env:ANDROID_KEY_PASS \ -p:AndroidSigningStorePass=env:ANDROID_STORE_PASS ```

Create a keystore once (keep it safe — losing it blocks future Play Store updates):

`bash keytool -genkeypair -v -keystore myrelease.keystore -alias myalias \ -keyalg RSA -keysize 2048 -validity 10000`

Artifacts land in `bin/Release/net10.0-android/publish/`.

### 16.3 Build a release iOS IPA

Publishing to the App Store requires an **Apple Distribution** certificate and an **App Store** provisioning profile.

`bash dotnet publish -f net10.0-ios -c Release \ -p:RuntimeIdentifier=ios-arm64 \ -p:CodesignKey="Apple Distribution: Your Name (TEAMID)" \ -p:CodesignProvision="My App Store Profile"`
```

Then upload with **Transporter** (Mac App Store) or `xcrun altool` / `notarytool` tooling.

---

17. Version compatibility matrix

The single most important rule: **match the Xcode version to your installed iOS/Mac Catalyst workload.** Each `dotnet/macios` release documents the Xcode it requires.

Component	Notes
.NET 10 SDK	GA, **LTS**. Check with `dotnet --version` (e.g. `10.0.401`).
MAUI workload	`sudo dotnet workload install maui`. Keep in step with the SDK.
.NET for iOS workload	Built for **one specific Xcode**. Example: workload set `10.0.401.1` ships iOS `27.0.10722` and requires **Xcode 27.0** (macOS 26.6+). Workload set `10.0.401` ships iOS `26.5.10318` and requires **Xcode 26.6**.
Xcode	Must satisfy the workload **and** macOS's minimum (e.g. Xcode 27.0 needs macOS 26.6+).
macOS	Keep current; recent Xcode needs a recent macOS.
VS Code	Latest stable.
.NET MAUI extension	Latest; requires an active C# Dev Kit subscription.

Authoritative sources — check before installing:

- .NET MAUI release versions (Xcode mapping): https://github.com/dotnet/maui/wiki/Release-Versions
- .NET for iOS/macOS releases (states required Xcode): https://github.com/dotnet/macios/releases
- .NET for Android releases: https://github.com/dotnet/android/releases

Typical error and fix:

`error: This version of .NET for iOS (26.5.10318) requires Xcode 26.6. The current version of Xcode is 27.0. Either install Xcode 26.6, or use a different version of .NET for iOS.`

Fix by installing the required Xcode (`xcodes install 26.6`) **or** by installing a workload set that matches your Xcode:

`bash dotnet --version sudo dotnet workload install maui --version 10.0.401.1 # matches Xcode 27.0`

After changing either, clean and rebuild:

`bash rm -rf bin obj dotnet restore && dotnet build`

---

18. Command cheat sheet

```bash
```

--- Versions / health ---

dotnet --version dotnet --info dotnet workload list xcodebuild -version xcode-select -p xcrun simctl list runtimes echo $JAVA_HOME echo $ANDROID_HOME "$ANDROID_HOME/platform-tools/adb" devices

--- Create / restore / build ---

dotnet new maui -n MyMauiApp dotnet restore dotnet build -f net10.0-ios -p:RuntimeIdentifier=iossimulator-arm64 dotnet build -f net10.0-android dotnet clean

--- Run ---

dotnet build -t:Run -f net10.0-ios -p:RuntimeIdentifier=iossimulator-arm64 dotnet build -t:Run -f net10.0-android

--- iOS simulator control ---

xcrun simctl list devices xcrun simctl boot "iPhone 17" open -a Simulator xcrun simctl shutdown all

--- Android emulator control ---

"$ANDROID_HOME/emulator/emulator" -list-avds "$ANDROID_HOME/emulator/emulator" -avd Pixel_API35 & "$ANDROID_HOME/platform-tools/adb" devices "$ANDROID_HOME/platform-tools/adb" logcat | grep -i mymauiapp

--- Workloads ---

sudo dotnet workload install maui sudo dotnet workload update sudo dotnet workload install maui --version ```

19. Troubleshooting

Symptom	Likely cause	Fix
`dotnet: command not found`	PATH not refreshed	Open a new terminal / `source ~/.zprofile`.
`This version of .NET for iOS (X) requires Xcode Y`	Xcode/workload mismatch	Install matching Xcode (`xcodes install`) or matching workload set (Section 17).
Xcode will not launch / missing components	Never launched Xcode or licence not accepted	`sudo xcodebuild -runFirstLaunch`; open Xcode and accept.
`xcode-select` points at CLT, not Xcode	CLT-only path	`sudo xcode-select --switch /Applications/Xcode.app/Contents/Developer`.
Android build: "Android SDK directory could not be found"	SDK path not set	Set `AndroidSdkDirectory` and `ANDROID_HOME` (Steps 6).
Android emulator very slow or will not start	Wrong ABI image	Use an `arm64-v8a` system image on Apple Silicon.
`adb: command not found`	platform-tools missing / not on PATH	`./sdkmanager --install "platform-tools"`; use the full path.
iOS build: "no provisioning profile" on device	Missing cert/profile	Create them in Xcode; register the device UDID (Step 12).
"Certificate already exists but cannot be found in the local Keychain"	Identity created on another machine	Export/import the `.p12` via Keychain Access.
Hot Reload does nothing	Some changes require a rebuild	Stop, `dotnet build`, then re-run.
Workload install fails on a preview SDK	Packs not on NuGet.org	Use the GA SDK, or add the correct preview feed.
Simulator will not boot	Corrupt simulator devices	`killall -9 com.apple.CoreSimulator.CoreSimulatorService`; delete `~/Library/Developer/CoreSimulator/Devices`; recreate.

20. Sources

- Install .NET MAUI tooling (VS Code) — Microsoft Learn: https://learn.microsoft.com/dotnet/maui/get-started/installation?view=net-maui-10.0
- Build your first .NET MAUI app — Microsoft Learn: https://learn.microsoft.com/dotnet/maui/get-started/first-app?view=net-maui-10.0
- Install .NET for Android dependencies — Microsoft Learn: https://learn.microsoft.com/dotnet/android/getting-started/installation/dependencies
- .NET for Android build targets — Microsoft Learn: https://learn.microsoft.com/dotnet/android/building-apps/build-targets
- .NET MAUI extension for VS Code — Visual Studio Marketplace: https://marketplace.visualstudio.com/items?itemName=ms-dotnettools.dotnet-maui
- .NET MAUI release versions (Xcode mapping) — GitHub wiki: https://github.com/dotnet/maui/wiki/Release-Versions
- .NET for iOS/macOS releases — GitHub: https://github.com/dotnet/macios/releases
- .NET 10 download — Microsoft: https://dotnet.microsoft.com/download/dotnet/10.0

> **Note:** Apple's toolchain and Microsoft's .NET workloads change frequently. This guide is accurate as of the date it was written. Always confirm the current Xcode-to-workload pairing in the GitHub sources above before installing.